Early Fault Detection in Electric Vehicles using Machine Learning

Project Report — SY B.Tech AIML

1) Background / Problem Statement

Electric Vehicles (EVs) are becoming the future of transportation all over the world, including in India. But one big challenge is keeping the main parts of the vehicle safe and reliable .especially the battery, motor, and cooling system. If something goes wrong in these parts, it leads mishaps

Traditional systems detect method reacts too late and cannot catch small changes that happen before a fault. Because of this, the warning often comes after the damage has already started.

To solve this problem, we developed an Early Fault Detection System for Electric Vehicles using Machine Learning. Our system is built in Python and keeps checking 19 sensor values in real time. These include battery temperature, voltage, current, motor speed, coolant flow, and smoke levels. It uses a trained classifier model from scikit-learn to predict faults early, before they become serious.

2) Reason for Topic Selection :

Members of my own family uses EV also we see lot of news like mishaps with EV battery, motor leading to fire/burning accidents. So I feel like everyone should be aware to such mishaps early by looking at its signs.

So everyone will be warned before such sudden mishaps. Hence I chose this topic.

3) Working of the System :

The system can detect 6 different types of faults with high accuracy and gives real-time alerts to the driver or fleet system. This helps in preventing failures and improving the safety of the vehicle.

My Python-based project is structured as a 4-stage machine learning:

- **Dataset collection**
- **Data preprocessing**
- **Model Training**
- **Real-Time Prediction.**

Stage 1 Dataset Collection :-

A realistic synthetic dataset of 2,000 records is generated using domain knowledge of EV engineering with help of AI. Each record contains 19 sensor features and is labelled with one of 7 conditions (classes). The dataset is saved as a CSV file.

- Normal condition
- Battery Overheat condition
- Battery Undervoltage condition
- Motor Overload condition
- Coolant Failure condition
- Fire Risk condition
- Cell Imbalance condition

Stage 2 EDA and Data Preprocessing :-

Before training the model, the raw dataset contained common real-world data quality issues(seen through EDA.py). All preprocessing was handled using Python's pandas library inside train_model.py. The following issues were identified and resolved:

Issue	Tool / Method Used	How It Was Handled
Missing Values	pandas — df.isnull(), df.fillna()	Numeric sensor columns filled with column median. Rows with missing target label dropped using df.dropna(subset=['fault_label']).
Duplicate Records	pandas — df.duplicated(), df.drop_duplicates()	All exact duplicate rows detected and removed using df.drop_duplicates(), keeping the first occurrence.
Outliers	pandas and ranging	calculated per sensor column. Values beyond range are cliped
Inconsistent Data	pandas — df.clip(), domain rules	Sensor values outside physical bounds (e.g. battery temp > 120C, SOC > 100%) clamped using df.clip(lower, upper) based on EV engineering limits.

Used EDA_verificate.py to ensure preprocessing worked

Stage 2 Model Training (train_model.py) :-

The cleaned dataset is loaded, and split into X,Y variable sets. Then model is trained and evaluated on test cases from same csv dataset to check accuracy of trained model:

Random Forest Classifier — gave 99% accuracy model

The trained models saved as .pkl files for use in prediction.py script . Feature importance scores are also computed to ensure accuracy

Stage 3 Real-Time Prediction (predict.py)

The prediction script loads the saved randomforest classifier model and accepts live sensor readings as input given to python code. It computes derived features, scales the input, and outputs the predicted fault class along with a confidence percentage. Actionable alerts are generated for

each fault type, ranging from informational warnings to critical stop-vehicle commands.

The dataset used in this project was synthetically generated using EV engineering domain knowledge and statistical distributions, as real-world labelled EV fault datasets are not publicly available at the required granularity. The model algorithm was selected as it consistently outperforms other algorithms for structured/tabular sensor data classification tasks.

The entire system is implemented in Python. Key libraries used are pandas and numpy for data handling, scikit-learn for preprocessing and Random Forest for classifier, and matplotlib , seaborn for data visualizing during EDA phase of pre processing

4) Advantages

- Detects faults early before they reach critical thresholds
- Identifies 6 distinct fault types with confidence scores
- Lightweight and fast XGBoost predictions execute in milliseconds, suitable for embedded systems.
- Modular pipeline — dataset, training, and prediction are independent and replaceable.
- Easily extendable to include new fault types or additional sensors.
- Can be integrated with real EV hardware

5) System Description

The system comprises the following :

Dataset

- 19 sensor input features: battery temperature, voltage, current, SOC, cell voltage min/max, motor temperature, RPM, motor current, coolant temperature, coolant flow, ambient temperature, cabin temperature, vibration, smoke PPM, speed, and etc features.
- 7 output classes: Normal, Battery Overheat, Battery Undervoltage, Motor Overload, Coolant Failure, Fire Risk, Cell Imbalance.

Training Module

- Importing nupy,pandas,scikit-learn
- Data loading and validation.
- Random Forest classifier training.
- Stratified train/test split .
- Accuracy evaluation and classification report generation.
- Model trained kept as .pkl files.

Prediction Module

- Accepts sensor readings as a Python inputs.
- Most important readings are taken.
- Applies saved pkl files trained model and runs model inference.
- Outputs fault label, fault name,a human-readable alert message.

6) Project Life Cycle :-

Phase	Description
Requirements	Define sensor inputs, fault types, alert outputs, and model accuracy targets.
Analysis	Study EV engineering literature to determine realistic sensor ranges for each fault condition.
Design	Design the 3-module pipeline architecture: dataset generation, model training, prediction.
Coding	Implement generate_dataset.py, train_model.py, and predict.py in Python.
Testing	Evaluate model accuracy (precision, recall, F1-score) on the test dataset using classification report.
Implementation	Deploy predict.py as a standalone prediction engine ready to be connected to real EV sensor feeds.

7) System Requirements :-

I. Hardware Requirements

- Laptop or PC — Windows 7 or higher
- Intel i3 processor or higher
- 4 GB RAM or higher
- 50 GB free disk space or higher

II. Software Requirements

- Python 3.8 or higher
- VS Code/other
- Libraries: pandas, numpy, scikit-learn,matplot,seaborn
- pip package manager in python environment (.venv)

8) Limitations / Disadvantages

- The dataset is synthetically generated and does not represent real-world sensor noise, sensor drift, or simultaneous multi-fault scenarios.
- Model accuracy on real EV sensor data may be lower than the 100% achieved on the synthetic test set.
- The system currently does not include a graphical user interface or live sensor integration.
- Fault thresholds are generalized and may need recalibration for specific EV models or manufacturers.

9) Application

- Can be integrated into EV onboard diagnostic systems for real-time fault detection.
- Useful for EV fleet management companies to monitor vehicle health remotely and reduce downtime.
- Can be deployed in EV charging stations to assess battery health before initiating a charging cycle.
- Applicable for predictive maintenance systems in EV manufacturing and quality testing pipelines.

10) References

- https://www.sciencedirect.com/topics/engineering/battery-fault-detection
- https://chatgpt.com for datasets reference and generation
- https://xgboost.readthedocs.io/en/stable/
- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

11) Code (Git-hub repository link: https://github.com/ayarekarpratik369/py_lab_project.git It has…) :-

1)EDA.py for exploratory data analysis

2)preprocess.py for preprocessing

3)preprocess report notes on preprocesses steps

4)EDA_verificate.py eda for verification of cleaning

5)train.py training data on cleaned dataset

6)predict.py to use .pkl and predict

7)requirements.txt about python tools used

8)README.md information about my folder

Predict.py:

```
import pandas as pd

import pickle



#Fault label map
```

```
fault_map = {
    0: "Normal",
    1: "Battery_Overheat",
    2: "Battery_Undervoltage",
    3: "Motor_Overload",
    4: "Coolant_Failure",
    5: "Fire_Risk",
    6: "Cell_Imbalance"
}


# Load model and top10 feature names
with open("model.pkl", "rb") as f:
    model = pickle.load(f)


with open("top10_features.pkl", "rb") as f:
    top10_features = pickle.load(f)


#  Take user input one by one
print("\n")
print("####  EV Fault Prediction — Enter Sensor Values  ####")
```

```
print("Type each sensor value and press Enter.\n")


user_input = {}


for feature in top10_features:
    while True:
        try:
            value = float(input(f"  Enter value for '{feature}': "))
            user_input[feature] = value
            break
        except ValueError:
            print(f"  Invalid input — please enter a number.")


# Predict
input_df = pd.DataFrame([user_input])
prediction = model.predict(input_df)[0]
fault_name = fault_map[prediction]


# Show result
```

```
print("\n" + "=" * 45)

print(f"  Predicted Fault : {fault_name}")

print(f"  Fault Label     : {prediction}")

print("=" * 45)
```

12) Output :-

```
####  EV Fault Prediction — Enter Sensor Values  ####
Type each sensor value and press Enter.

  Enter value for 'battery_temp_C': 45
  Enter value for 'cell_voltage_delta': 34
  Enter value for 'motor_temp_C': 76
  Enter value for 'temp_delta_battery_ambient': 54
  Enter value for 'cell_voltage_min_V': 56
  Enter value for 'thermal_runaway_index': 23
  Enter value for 'vibration_g': 43
  Enter value for 'smoke_ppm': 90
  Enter value for 'coolant_flow_Lpm': 45
  Enter value for 'coolant_temp_C': 34

=============================================
  Predicted Fault : Fire_Risk
  Fault Label     : 5
=============================================

d:\pylab_mini_project>
```

13) Video Link of Execution :-

https://drive.google.com/file/d/17vnGJBNEqk0wG3SP8xROrPxsOfeSh_ga/view?usp=drive_link